ChatGPT

Binary Search Playlist Problem Solutions (Aditya Verma)

Below we compile the problems from the Binary Search playlist [1] [2], categorized by topic. Each problem includes a brief description, a link to the problem on LeetCode or GeeksforGeeks (if available), a clean C++14 solution, and an analysis of time and space complexity.

Basic Binary Search Problems

Binary Search (Ascending Order)

Description: Given a sorted array in ascending order and a target value, find the index of the target in the array. If the target is not found, return -1. This is the standard binary search problem [3].

Solution (C++14):

```
int binarySearchAsc(const vector<int>& arr, int target) {
    int low = 0, high = arr.size() - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) {
            return mid;
        } else if (arr[mid] < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return -1;  // not found
}
```

- **Time Complexity:** O(log N) – binary search halves the array range at each step.
- **Space Complexity:** O(1) – only a few variables are used regardless of input size.

Relevant Link: LeetCode 704 (Binary Search) – Problem

Binary Search (Descending Order)

Description: Given a sorted array in *descending* order, find the index of a target value. This is a slight variation where the comparison direction is reversed (also known as order-agnostic binary search [4]).

Solution (C++14):

```cpp
int binarySearchDesc(const vector<int>& arr, int target) {
    int low = 0, high = arr.size() - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) {
            return mid;
        } else if (arr[mid] > target) {
            low = mid + 1;  // move right in a descending array
        } else {
            high = mid - 1;
        }
    }
    return -1;
}
```

- **Time Complexity:** O(log N) – binary search on a sorted array (descending).
- **Space Complexity:** O(1).

Relevant Link: GeeksforGeeks – Order-Agnostic Binary Search

Order-Agnostic Binary Search

Description: Given a sorted array that could be in ascending *or* descending order, find the index of a target value. The algorithm first checks the order of the array (by comparing the first and last elements) and then performs binary search accordingly [4].

Solution (C++14):

```cpp
int orderAgnosticBinarySearch(const vector<int>& arr, int target) {
    if (arr.empty()) return -1;
    bool asc = arr.front() < arr.back();  // determine sort order
    int low = 0, high = arr.size() - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) {
            return mid;
        }
        if (asc) {
            // array is ascending
            if (arr[mid] < target) low = mid + 1;
            else high = mid - 1;
        } else {
            // array is descending
            if (arr[mid] < target) high = mid - 1;
            else low = mid + 1;
        }
    }
```

```
    return -1;
}
```

- **Time Complexity:** O(log N) – one binary search pass after checking order.
- **Space Complexity:** O(1).

Relevant Link: GeeksforGeeks – Order-Agnostic Binary Search

Sorted Array Search Problems

First and Last Occurrence in Sorted Array

Description: Given a sorted array (ascending) that may contain duplicate values, find the first and last index of a given target value 3. If the target is not present, return -1 for both first and last positions. (LeetCode variant: "Find First and Last Position of Element in Sorted Array").

Solution (C++14): We perform two binary searches – one to find the first occurrence and one to find the last occurrence of the target.

```
pair<int,int> findFirstAndLast(const vector<int>& arr, int target) {
    int n = arr.size();
    int first = -1, last = -1;
    // Find first occurrence (leftmost index)
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] >= target) {
            if (arr[mid] == target) first = mid;
            high = mid - 1;  // continue search on left half
        } else {
            low = mid + 1;
        }
    }
    // Find last occurrence (rightmost index)
    low = 0; high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] <= target) {
            if (arr[mid] == target) last = mid;
            low = mid + 1;  // continue search on right half
        } else {
            high = mid - 1;
        }
    }
    return {first, last};
}
```

- **Time Complexity:** O(log N) – each binary search takes O(log N), so two searches are O(2·log N) = O(log N).
- **Space Complexity:** O(1).

Relevant Link: GeeksforGeeks – First and Last Occurrences of x [3]

Count Occurrences in Sorted Array

Description: Given a sorted array and a target value, count the number of times the target appears in the array [5]. If the target is not present, the count is 0. This can be solved by finding the first and last occurrence and computing the difference.

Solution (C++14): We reuse the first/last occurrence logic above and then compute the count.

```
int countOccurrences(const vector<int>& arr, int target) {
    auto bounds = findFirstAndLast(arr, target);
    int first = bounds.first, last = bounds.second;
    if (first == -1) {
        return 0;
    }
    return last - first + 1;
}
```

- **Time Complexity:** O(log N) – based on binary search for first and last positions.
- **Space Complexity:** O(1).

Relevant Link: GeeksforGeeks – Number of Occurrence [5]

Floor of an Element in a Sorted Array

Description: Given a sorted array and a value `x`, find the **floor** of `x` in the array. The floor of `x` is the largest element in the array that is $\leq$ `x`. If no such element exists (i.e., all elements are greater than `x`), return -1 [6].

Solution (C++14): We can modify binary search to track the floor. As we search, if an element is <= `x`, it becomes a candidate for floor.

```
int findFloor(const vector<int>& arr, int x) {
    int low = 0, high = arr.size() - 1;
    int floorIndex = -1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == x) {
            return mid;  // exact match is the floor itself
        } else if (arr[mid] < x) {
            floorIndex = mid;   // arr[mid] is a new floor candidate
            low = mid + 1;      // move right to find a possibly larger floor
        } else {  // arr[mid] > x
```

```
            high = mid - 1;     // move left to find smaller elements
        }
    }
    return floorIndex;
}
```

- **Time Complexity:** O(log N).
- **Space Complexity:** O(1).

Relevant Link: GeeksforGeeks – Floor in a Sorted Array [6]

Ceil of an Element in a Sorted Array

Description: Given a sorted array and a value `x`, find the **ceil** of `x` in the array. The ceil of `x` is the smallest element in the array that is $\geq$ `x`. If no such element exists (all elements are smaller than `x`), return -1 [7].

Solution (C++14): This is the mirror of the floor problem. We track the ceil candidate while binary searching.

```
int findCeil(const vector<int>& arr, int x) {
    int low = 0, high = arr.size() - 1;
    int ceilIndex = -1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == x) {
            return mid;  // exact match is the ceil itself
        } else if (arr[mid] < x) {
            low = mid + 1;  // need a larger element
        } else {  // arr[mid] > x
            ceilIndex = mid;
            high = mid - 1;   // move left to find a smaller or equal element
        }
    }
    return ceilIndex;
}
```

- **Time Complexity:** O(log N).
- **Space Complexity:** O(1).

Relevant Link: (No direct LeetCode problem; similar logic discussed on GeeksforGeeks)

Minimum Difference Element in Sorted Array

Description: Given a sorted array and a target value, find the element in the array which has the minimum absolute difference to the target [8]. If the array contains the target, then that target is the result. Otherwise, it will be one of the neighbors around where the target would be inserted.

Solution (C++14): We first perform binary search to find the position where the target would fit (using floor/ceil). Then we compare the candidate just smaller and just larger than the target to see which is closer.

```
int minDiffElement(const vector<int>& arr, int target) {
    int n = arr.size();
    if (n == 0) return -1;
    // If target is out of range of array values, return the boundary element
    if (target <= arr.front()) return arr.front();
    if (target >= arr.back()) return arr.back();
    // Binary search to find floor and ceil positions
    int low = 0, high = n - 1;
    int floorIdx = -1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) {
            return arr[mid];  // target found
        } else if (arr[mid] < target) {
            floorIdx = mid;
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    // floorIdx will hold index of largest element < target
    // low will be index of smallest element > target (after loop, low ==
floorIdx+1)
    int ceilIdx = low;
    // Compare which of arr[floorIdx] or arr[ceilIdx] is closer to target
    if (floorIdx >= 0 && ceilIdx < n) {
        int diff1 = target - arr[floorIdx];
        int diff2 = arr[ceilIdx] - target;
        return (diff1 <= diff2 ? arr[floorIdx] : arr[ceilIdx]);
    }
    // fallback (shouldn't normally hit these if array not empty and target
within range)
    if (floorIdx >= 0) return arr[floorIdx];
    if (ceilIdx < n) return arr[ceilIdx];
    return -1;
}
```

- **Time Complexity:** O(log N) – one binary search, then constant time comparisons.
- **Space Complexity:** O(1).

Relevant Link: (Discussed in multiple interview problem sets, similar to *Find element with minimum difference* on GeeksforGeeks)

Advanced Binary Search in Arrays

Search in Rotated Sorted Array

Description: Given an array that was originally sorted in ascending order but then rotated an unknown number of times, find the index of a target value. If not found, return -1. (This is a classic LeetCode problem "Search in Rotated Sorted Array" [9].)

Solution (C++14): We can modify binary search by first determining which half of the array is sorted, then deciding which half to search in:

```
int searchInRotated(const vector<int>& arr, int target) {
    int low = 0, high = arr.size() - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) {
            return mid;
        }
        // Check if left half is sorted
        if (arr[low] <= arr[mid]) {
            if (target >= arr[low] && target < arr[mid]) {
                high = mid - 1;
            } else {
                low = mid + 1;
            }
        }
        // Otherwise, right half must be sorted
        else {
            if (target > arr[mid] && target <= arr[high]) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
    }
    return -1;
}
```

- **Time Complexity:** O(log N) – binary search with additional checks.
- **Space Complexity:** O(1).

Relevant Link: LeetCode 33 – Search in Rotated Sorted Array

Rotation Count of Sorted Array

Description: Given an array of distinct numbers sorted in ascending order and then rotated, find the number of times the array has been rotated. This is effectively the index of the smallest element (because each rotation puts the former smallest at the front). For example, array `[15, 18, 2, 3, 6, 12]` has been rotated 2 times (smallest element `2` is at index 2) [9].

Solution (C++14): We find the index of the minimum element using binary search. One approach is to look for the inflection point where an element is smaller than its previous element. Another approach is to modify binary search conditions as shown:

```
int rotationCount(const vector<int>& arr) {
    int low = 0, high = arr.size() - 1;
    int n = arr.size();
    while (low <= high) {
        if (arr[low] <= arr[high]) {
            // subarray [low..high] is sorted, so low is index of smallest
            return low;
        }
        int mid = low + (high - low) / 2;
        int next = (mid + 1) % n;
        int prev = (mid - 1 + n) % n;
        // Check if mid is the smallest
        if (arr[mid] <= arr[next] && arr[mid] <= arr[prev]) {
            return mid;
        }
        // Decide which half to choose
        if (arr[mid] >= arr[low]) {
            // left half is sorted, so pivot must be in right half
            low = mid + 1;
        } else {
            // right half is sorted, so pivot is in left half
            high = mid - 1;
        }
    }
    return 0;
}
```

- **Time Complexity:** O(log N).
- **Space Complexity:** O(1).

Relevant Link: GeeksforGeeks – Rotation Count in Rotated Sorted Array

Search in Nearly Sorted Array

Description: In a nearly sorted array, each element may be misplaced by at most one position from where it would be in a fully sorted order. For example, in a nearly sorted array, an element that should be at index `i` might be at `i-1`, `i`, or `i+1`. Given such an array and a target, find the index of the target. If not found, return -1.

Solution (C++14): We can modify binary search to account for the neighboring elements. At each mid, check not only `arr[mid]` but also `arr[mid-1]` and `arr[mid+1]` for the target (if those indices are in range), then proceed as in normal binary search by comparing to decide which half to search.

```
int searchNearlySorted(const vector<int>& arr, int target) {
    int low = 0, high = arr.size() - 1;
```

```
    while (low <= high) {
        int mid = low + (high - low) / 2;
        // Check mid and its neighbors for target
        if (mid >= 0 && mid < arr.size() && arr[mid] == target) {
            return mid;
        }
        if (mid-1 >= low && arr[mid-1] == target) {
            return mid-1;
        }
        if (mid+1 <= high && arr[mid+1] == target) {
            return mid+1;
        }
        // If target is smaller, ignore right part
        if (arr[mid] > target) {
            high = mid - 2;  // skip the neighbor since we already checked
mid-1
        } else {
            low = mid + 2;   // skip neighbor mid+1 as checked
        }
    }
    return -1;
}
```

- **Time Complexity:** O(log N) on average. (In the worst case, the constant factor is higher due to checking neighbors, but it remains proportional to binary search steps.)
- **Space Complexity:** O(1).

Relevant Link: (Variant discussed in various coding interviews; similar approach in GeeksforGeeks article for nearly sorted array search)

Search in Infinite Sorted Array (First 1 in Infinite Binary Sorted Array)

Description: Suppose you have an infinite sorted array (conceptually unbounded) of 0s followed by 1s (all 0's come before all 1's). Find the index of the first `1` in this infinite sorted array `10`. More generally, for an infinite sorted array of unknown length, we can find a target by expanding the search range exponentially.

Solution (C++14): We first expand the range exponentially until we find a `1`, then perform binary search in that range.

```
int indexOfFirstOneInfinite(const vector<int>& arr) {
    // We simulate an "infinite" array by using the given vector (which
should contain 0s then 1s).
    int low = 0, high = 1;
    // Expand range until a '1' is found or high exceeds array size
    while (high < arr.size() && arr[high] == 0) {
        low = high;
        high *= 2;
        if (high >= arr.size()) high = arr.size() - 1;
    }
```

```
    // Now perform binary search between low and high for the first 1
    int firstOneIndex = -1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == 1) {
            firstOneIndex = mid;
            high = mid - 1;  // look to the left for an earlier 1
        } else {  // arr[mid] == 0
            low = mid + 1;
        }
    }
    return firstOneIndex;
}
```

- **Time Complexity:** O(log N) – the range expansion grows exponentially (doubling), and the binary search in the found range is O(log N). Overall complexity is O(log N). [11]
- **Space Complexity:** O(1).

Relevant Link: GeeksforGeeks – Index of first 1 in infinite sorted array of 0s and 1s [11]

Bitonic Array Problems

(A bitonic array is one that is first strictly increasing then strictly decreasing. It has a single "peak" element which is the maximum.)

Peak Element in Unsorted Array

Description: A **peak element** in an array is one that is strictly greater than its neighbors. Given an array (not necessarily sorted, and adjacent values are not equal), find the index of any one peak element [12] [13]. If the array has multiple peaks, any peak index can be returned. (LeetCode 162: Find Peak Element)

Solution (C++14): We can find a peak in O(log N) by binary search: compare middle with neighbors to decide which side has a peak. If `arr[mid]` is less than `arr[mid+1]`, then a peak must lie on the right side; otherwise it lies on the left side (including mid).

```
int findPeak(const vector<int>& arr) {
    int low = 0, high = arr.size() - 1;
    while (low < high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] < arr[mid + 1]) {
            // there's an increasing trend, so peak is to the right
            low = mid + 1;
        } else {
            // arr[mid] > arr[mid+1], so a peak is at mid or to the left
            high = mid;
        }
    }
    // low == high is the peak index
```

```
    return low;
}
```

- **Time Complexity:** O(log N) – binary search approach to peak finding [14].
- **Space Complexity:** O(1).

Relevant Link: LeetCode 162 – Find Peak Element

Maximum Element in Bitonic Array (Bitonic Point)

Description: Given a bitonic array (strictly increasing then strictly decreasing), find the **maximum element** (also known as the *bitonic point* of the array). This element is greater than all others and is the point where the sequence changes direction [15].

Solution (C++14): The maximum element in a bitonic array can be found with a modified binary search that looks for the inflection point where `arr[mid] > arr[mid+1]` (peak condition). This is similar to finding a peak, but here the peak is unique.

```
int findBitonicMax(const vector<int>& arr) {
    int low = 0, high = arr.size() - 1;
    while (low < high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] < arr[mid + 1]) {
            // still increasing, go right
            low = mid + 1;
        } else {
            // mid is greater than next, so peak is at mid or to the left
            high = mid;
        }
    }
    return arr[low];  // low is index of max element
}
```

- **Time Complexity:** O(log N).
- **Space Complexity:** O(1).

Relevant Link: GeeksforGeeks – Bitonic Point (Maximum in bitonic array)

Search in Bitonic Array

Description: Given a bitonic array (first increasing then decreasing) of **distinct** elements and a target value, determine the index of the target in the array, or return -1 if it is not present [16]. We need to account for the increase-then-decrease property while searching.

Solution (C++14): First, find the index of the maximum element (peak). Then perform binary search on the increasing subarray (left of the peak) and on the decreasing subarray (right of the peak) separately.

```cpp
int searchInBitonic(const vector<int>& arr, int target) {
    int n = arr.size();
    // 1. Find peak index
    int low = 0, high = n - 1;
    int peak = 0;
    while (low < high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] < arr[mid+1]) {
            low = mid + 1;
        } else {
            high = mid;
        }
    }
    peak = low;
    // 2. Binary search in the increasing part [0..peak]
    low = 0; high = peak;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    // 3. Binary search in the decreasing part [peak+1..n-1]
    low = peak + 1; high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] > target) low = mid + 1;   // now array is decreasing
        else high = mid - 1;
    }
    return -1;
}
```

- **Time Complexity:** O(log N) – finding the peak is O(log N), and each binary search on the two halves is O(log N). Overall O(log N). [17]
- **Space Complexity:** O(1).

Relevant Link: GeeksforGeeks – Find an element in Bitonic array [16]

Matrix Search Problem

Search in Row-wise and Column-wise Sorted Matrix

Description: Given a 2D matrix sorted in ascending order in each row and each column (each row is sorted left to right, and each column is sorted top to bottom), determine whether a target value exists in the matrix. This is sometimes known as the "staircase search" problem [18]. Example: In matrix

```
[ [1,  4,  7],
  [2,  5,  9],
  [3,  6, 10] ]
```

each row and column is sorted.

Solution (C++14): A common efficient strategy is to start from the top-right corner and move left or down depending on comparisons (since going left decreases value, going down increases value).

```cpp
bool searchMatrix(const vector<vector<int>>& matrix, int target) {
    if (matrix.empty() || matrix[0].empty()) return false;
    int n = matrix.size();
    int m = matrix[0].size();
    int row = 0, col = m - 1;  // start at top-right corner
    while (row < n && col >= 0) {
        if (matrix[row][col] == target) {
            return true;
        } else if (matrix[row][col] > target) {
            col--;      // move left to smaller values
        } else {
            row++;      // move down to larger values
        }
    }
    return false;
}
```

- **Time Complexity:** O(N + M) – at most one pass through a row or column (each step moves one row down or one column left). For an N×M matrix, worst case we move O(N+M) steps. This is efficient for large matrices.
- **Space Complexity:** O(1).

Relevant Link: LeetCode 240 – Search a 2D Matrix II

Binary Search on Answer (Partition Problems)

(These problems use binary search on a solution space of answers. We guess a potential answer and check feasibility with a given predicate, adjusting the search range based on the check result [19].)

Allocate Minimum Number of Pages (Book Allocation Problem)

Description: Given an array of book page counts and an integer `m` (number of students), allocate books to the students such that each student gets a contiguous segment of books and the maximum pages assigned to any student is minimized [20]. We need to find that minimum possible value of the maximum pages. This problem is known as the Book Allocation problem, and it's analogous to the Painters Partition problem. (It is also comparable to LeetCode "Split Array Largest Sum".)

Solution (C++14): We use binary search on the range of possible answers (minimum and maximum pages possible). For a given mid value (guess for the maximum pages), we check if it's possible to

allocate books to `m` students without any student having more than `mid` pages. If it's possible, we try a lower value; if not, we increase the value.

```
bool canAllocate(const vector<int>& pages, int m, long long maxPages) {
    int studentCount = 1;
    long long currentSum = 0;
    for (int p : pages) {
        if (p > maxPages) return
false;  // a single book has more pages than allowed
        if (currentSum + p > maxPages) {
            // allocate to next student
            studentCount++;
            currentSum = p;
            if (studentCount > m) return false;
        } else {
            currentSum += p;
        }
    }
    return true;
}

long long allocateMinPages(const vector<int>& pages, int m) {
    long long low = 0;
    long long high = 0;
    for (int p : pages) {
        high += p;
        low = max(low, (long long)p);
    }
    long long result = high;
    while (low <= high) {
        long long mid = low + (high - low) / 2;
        if (canAllocate(pages, m, mid)) {
            result = mid;
            high = mid - 1;
        } else {
            low = mid + 1;
        }
    }
    return result;
}
```

- **Time Complexity:** O(N * log S), where N is number of books and S is the sum of all pages. The binary search runs in O(log S) steps, and for each step we potentially iterate through all books to check feasibility. [21]
- **Space Complexity:** O(1).

Relevant Link: GeeksforGeeks – Allocate minimum number of pages (Book Allocation) [20]

Aggressive Cows (Maximize Minimum Distance)

Description: Given an array of stall positions (unsorted) and an integer `k` (number of cows), place the `k` cows in the stalls such that the minimum distance between any two cows is maximized [22]. This is a classic binary search on answer problem: we guess a distance and check if cows can be placed with at least that separation.

Solution (C++14): We first sort the stall positions. Then binary search on the possible distance (low = 1, high = max(pos) - min(pos)). For each candidate distance, greedily place cows and see if all `k` can be placed.

```
bool canPlaceCows(const vector<int>& stalls, int k, int dist) {
    int count = 1;
    int last_pos = stalls[0];
    for (int i = 1; i < stalls.size(); ++i) {
        if (stalls[i] - last_pos >= dist) {
            count++;
            last_pos = stalls[i];
            if (count == k) return true;
        }
    }
    return false;
}

int aggressiveCows(vector<int>& stalls, int k) {
    sort(stalls.begin(), stalls.end());
    int low = 1;
    int high = stalls.back() - stalls.front();
    int best = 0;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (canPlaceCows(stalls, k, mid)) {
            best = mid;      // mid distance is possible, try for larger
            low = mid + 1;
        } else {
            high = mid - 1;  // mid distance not possible, try smaller
        }
    }
    return best;
}
```

- **Time Complexity:** O(N log D), where N is number of stalls and D is the search space for distance (max-min). Sorting takes O(N log N). The feasibility check for each mid takes O(N). [22]
- **Space Complexity:** O(1) (ignoring sort space).

Relevant Link: GeeksforGeeks – Aggressive Cows [22] (SPOJ Aggressive Cows problem)

---

1 2 GitHub - tiyasha99/Binary-Search-By-Aditya-Verma
https://github.com/tiyasha99/Binary-Search-By-Aditya-Verma

3 First and Last Occurrences | Practice - GeeksforGeeks
https://www.geeksforgeeks.org/problems/first-and-last-occurrences-of-x3116/1

4 Order-Agnostic Binary Search - GeeksforGeeks
https://www.geeksforgeeks.org/dsa/order-agnostic-binary-search/

5 Number of occurrence | Practice - GeeksforGeeks
https://www.geeksforgeeks.org/problems/number-of-occurrence2259/1

6 Floor in a Sorted Array - GeeksforGeeks
https://www.geeksforgeeks.org/dsa/floor-in-a-sorted-array/

7 Find floor and ceil in an unsorted array - GeeksforGeeks
https://www.geeksforgeeks.org/dsa/find-floor-ceil-unsorted-array/

8 Minimum Difference Element in sorted array - Discuss - LeetCode
https://leetcode.com/discuss/study-guide/3397027/Minimum-Difference-Element-in-sorted-array

9 Array for coding Round - YouTube
https://m.youtube.com/playlist?list=PLVcKgSGeiTHnLkDm-7cuqidfr8kEr_bZG

10 Arrays Archives - GeeksforGeeks
https://www.geeksforgeeks.org/category/dsa/data-structures/c-arrays/page/243/

11 Find the index of first 1 in an infinite sorted array of 0s and 1s
https://www.geeksforgeeks.org/dsa/find-index-first-1-infinite-sorted-array-0s-1s/

12 13 Peak Element in Array - GeeksforGeeks
https://www.geeksforgeeks.org/dsa/find-a-peak-in-a-given-array/

14 Find the Peak Element | C++ Placement Course | Lecture 29.5
https://www.youtube.com/watch?v=KWDUOForS2s

15 16 Find an element in Bitonic array - GeeksforGeeks
https://www.geeksforgeeks.org/dsa/find-element-bitonic-array/

17 18 Find maximum element in Bitonic Array - YouTube
https://www.youtube.com/watch?v=BrrZL1RDMwc

18 Search in Bitonic Array! - InterviewBit
https://www.interviewbit.com/problems/search-in-bitonic-array/

19 16 Binary Search on Answer Concept - YouTube
https://www.youtube.com/watch?v=IZP_8-JZqhM

20 Allocate Minimum Number Of Pages - YouTube
https://www.youtube.com/watch?v=2JSQIhPcHQg

21 XitizVerma/Data-Structures-and-Algorithms-Advanced - GitHub
https://github.com/XitizVerma/Data-Structures-and-Algorithms-Advanced

22 Aggressive Cows | Practice - GeeksforGeeks
https://www.geeksforgeeks.org/problems/aggressive-cows/1